

# Container und virtuelle Maschinen – Isolationsmechanismen, Bedrohungsmodelle und Sicherheitsanalyse

IT-Sicherheit Übung – Thema 12, Hochschule Karlsruhe

1<sup>st</sup> Simon Klaes  
Hochschule Karlsruhe  
Karlsruhe, Deutschland  
klsi1012@h-ka.de

2<sup>nd</sup> Radek Pekul  
Hochschule Karlsruhe  
Karlsruhe, Deutschland  
pera1011@h-ka.de

3<sup>rd</sup> Joel Maxeiner  
Hochschule Karlsruhe  
Karlsruhe, Deutschland  
majo1064@h-ka.de

4<sup>th</sup> Pius Tebbe  
Hochschule Karlsruhe  
Karlsruhe, Deutschland  
tepi1011@h-ka.de

**Zusammenfassung**—Container und virtuelle Maschinen (VMs) sind die zentralen Bausteine moderner IT-Infrastrukturen, unterscheiden sich aber grundlegend in ihrer Isolationsstärke. Diese Arbeit vergleicht beide Konzepte hinsichtlich ihrer Sicherheitseigenschaften: Sie erläutert die fundamentalen Isolationsmechanismen (Namespaces, cgroups, Hypervisor-Virtualisierung), demonstriert in einer Live-Demo den Normalbetrieb beider Technologien sowie typische Fehlkonfigurationen, und diskutiert reale Escape-Szenarien und Best Practices. Ziel ist es, fundiert zu zeigen, wann welche Technologie sicherheitstechnisch angemessen ist.

**Index Terms**—Container, Docker, virtuelle Maschine, Hypervisor, Namespaces, cgroups, Isolation, Container-Escape, IT-Sicherheit

## I. GRUNDLAGEN DER ISOLATION

### A. Motivation und Bedrohungsmodell

Sobald mehrere Workloads auf derselben Hardware laufen, stellt sich die Frage, *welche Komponente bei einem Ausbruch versagen darf*. Beim **Container-Bedrohungsmodell** wird angenommen, dass der Angreifer bereits Code im Container ausführt – etwa über eine kompromittierte Webanwendung oder ein böses Image – und versucht, den Host oder Nachbarcontainer zu übernehmen. Beim **VM-Bedrohungsmodell** wird sogar angenommen, dass der Gast vollständig kompromittiert ist (Root im Gast-Kernel); dennoch sollen Host und andere Gäste sicher bleiben. Container schützen damit *vor versehentlichen Konflikten zwischen vertrauenswürdigen Workloads*, VMs dagegen *vor bösen Mandanten* und sind deshalb Standard für Multi-Tenant-Clouds wie AWS EC2 oder Azure VMs [1].

### B. Virtualisierungskonzepte: Hypervisor-Typen

Eine VM simuliert einen kompletten Computer mit eigenem Kernel, eigener Hardware-Sicht und eigenem Bootloader. Der Hypervisor ist die Schicht, die diese Trennung erzwingt:

- **Typ 1 (Bare-Metal)**: läuft direkt auf der Hardware, ohne Host-OS dazwischen. Beispiele: KVM, Xen, VMware ESXi, Microsoft Hyper-V. Geringe Trusted Computing Base (TCB) und produktiv bevorzugt.

- **Typ 2 (Hosted)**: läuft als Anwendung auf einem Betriebssystem. Beispiele: VirtualBox, VMware Workstation, Parallels Desktop, UTM.

Hardware-Erweiterungen (Intel VT-x, AMD-V, ARM Virtualization Extensions) führen einen privilegierten CPU-Modus ein („Ring –1“ bzw. EL2 auf ARM). Privilegierte Operationen des Gasts lösen einen *VM Exit* aus, den der Hypervisor behandelt. EPT/NPT übersetzt Gast-physikalische zu Host-physikalischen Adressen in Hardware. Die Schnittstelle zwischen Gast und Host besteht damit nur aus wenigen Hypercalls und paravirtualisierten Geräten (virtio) – eine deutlich kleinere Angriffsfläche als das Linux-Syscall-Interface [2].

### C. Container-Grundlagen: Namespaces, Cgroups, Kernel-Sharing

Container sind technisch **keine eigene Technologie**, sondern eine Kombination bestehender Linux-Kernel-Features. Sie sind im Wesentlichen isolierte Prozesse, die denselben Kernel mit dem Host teilen.

**Namespaces** schaffen pro Container eine eigene Sicht auf Kernel-Ressourcen: PID (eigene Prozessbäume), NET (eigener Netzwerkstack), MNT (eigene Mount-Points), IPC, UTS (Hostname), USER (UID-Mapping), CGROUP und TIME. Namespaces beschränken *Sichtbarkeit*, nicht welche Syscalls ausgeführt werden dürfen.

**Control Groups (cgroups)** sind orthogonal dazu und beschränken *Ressourcenverbrauch* (CPU, Speicher, Block-I/O, Anzahl Prozesse). Ohne cgroups könnte ein einzelner Container per Fork-Bombe das Host-System lahmlegen.

Ergänzend reduzieren weitere Mechanismen die Angriffsfläche: **Capabilities** zerlegen die Root-Rechte in ca. 40 Einzelrechte (Container erhalten standardmäßig nur ca. 14), **Seccomp-BPF** filtert Syscalls (Docker blockiert per Default ca. 60 von ca. 330), und **LSMs** wie AppArmor oder SELinux setzen Mandatory Access Control durch.

#### D. Vertrauensgrenzen und Vergleich

Der entscheidende Unterschied ist die Frage, was bei einem Ausbruch versagen muss. Bei Containern ist die Vertrauensgrenze der **geteilte Linux-Kernel** – jede ausnutzbare Kernel-Schwachstelle ist potenziell ein Container-Escape. Die TCB umfasst ca. 30 Millionen Zeilen Code (gesamter Kernel inkl. Treiber, Syscall-Interface, Dateisystemen). Bei VMs ist die Vertrauensgrenze der **Hypervisor**; die TCB im Hot-Path liegt z. B. bei KVM bei rund 100 000 Zeilen Code – zwei bis drei Größenordnungen weniger Angriffsfläche [1]. Tabelle I fasst die wesentlichen Dimensionen zusammen.

Tabelle I  
VERGLEICH DER ISOLATIONSGARANTIEN

| Dimension        | Container        | VM                    |
|------------------|------------------|-----------------------|
| Isolationsgrenze | Kernel (geteilt) | Hardware (Hypervisor) |
| Boot-Zeit        | Millisekunden    | Sekunden bis Minuten  |
| Overhead         | MB               | Hunderte MB bis GB    |
| Dichte/Host      | Hunderte         | Dutzende              |
| Angriffsfläche   | ~330 Syscalls    | Hypercalls + Geräte   |
| TCB              | ~30 Mio. LOC     | ~100k LOC             |
| Multi-Tenancy    | Unzureichend     | Standard              |

## II. IMPLEMENTIERUNG UND NORMALBETRIEB

*[Abschnitt von Person B: Setup des Demo-Dienstes (Python HTTP-Server) sowohl als Docker-Container als auch in einer VirtualBox/UTM-VM, Ressourcenisolierung mit docker stats und htop, Netzwerkvergleich Container-Bridge vs. NAT/Bridged, Analyse von ps, top, /proc, Fazit zur vorgeesehenen Isolation.]*

## III. FEHLKONFIGURATIONEN UND SCHWACHSTELLEN

*[Abschnitt von Person C: Demonstration und Analyse typischer Fehlkonfigurationen. Container: -privileged, gemountete Host-Volumes (-v /etc:/host-etc), deaktiviertes Seccomp-Profil (-security-opt seccomp=unconfined), gemounteter Docker-Socket. VM: unsichere Bridged-Konfiguration, Shared Folders, aktive Komfort-Geräte. Für jede Fehlkonfiguration: welche Isolationsgrenze bricht?]*

## IV. ESCAPE-SZENARIEN UND BEST PRACTICES

*[Abschnitt von Person D: Container-Escapes mit CVE-Beispielen (CVE-2019-5736 runc, CVE-2022-0492 cgroups v1, Dirty Pipe CVE-2022-0847), VM-Escapes (VENOM CVE-2015-3456, VirtualBox-CVEs), Side-Channel-Angriffe (Spectre/Meltdown). Best Practices: Rootless Container, minimale Images, seccomp, read-only Filesystem, Netzwerksegmentierung, Hypervisor-Härtung. Gesamtfazit: wann Container, wann VM?]*

## HINWEIS ZUR KI-NUTZUNG

Bei der Recherche und der Erstellung dieses Dokuments wurden KI-gestützte Werkzeuge (Anthropic Claude) zur Strukturierung von Inhalten, zum Erklären von Fachbegriffen und zur sprachlichen Glättung eingesetzt. Sämtliche inhaltlichen Aussagen wurden von den Autorinnen und Autoren geprüft und gegen die Lehrgangs- und Referenzquellen abgeglichen.

## LITERATUR

- [1] Systemsicherheits-Lehrgang, „Container vs. Virtuelle Maschinen – Ein tiefgehender Lehrgang“, Hochschule Karlsruhe, 2026.
- [2] „Container und Virtuelle Maschinen – Isolationsmechanismen, Bedrohungsmodelle und Sicherheitsanalyse“, internes Skript, Hochschule Karlsruhe, 2026.
- [3] MITRE, CVE-2019-5736 (runc-Container-Breakout), 2019.
- [4] M. Kellermann, „The Dirty Pipe Vulnerability (CVE-2022-0847)“, 2022.
- [5] CrowdStrike, „VENOM Vulnerability (CVE-2015-3456)“, 2015.